Plan 07 — Is detection real?

An offline-first re-test of the memory-signal spine

Comprehensive experimental-design handout · June 2026

1. Summary

Plan 07 builds a behaviour recogniser from a single host-side memory signal. Watching a VM from the outside -- no agent inside the guest -- the system places a running workload into a known **behaviour family** (memory-sweep-like, IO-like, file-processing-like, ...) or flags it as **novel**. That recogniser is the centrepiece and the claim. Threat-flagging is a *downstream consequence* of it: a behaviour that matches a known threat-shaped family is flagged for review. We build (b) the recogniser; (a) detection rides on top of it.

What already works, and what is open. We can already identify a workload's behaviour *when its kind has been seen before* -- instance and family identification reach ~0.97 on the optimistic (seen-before) split. That much is established. What Plan 07 answers is the open part: does a per-family 'normal-behaviour' model generalise to an *unseen* member of its family; do families even have a **coherent memory signature** (or does our taxonomy not match the data); can the system flag a **novel** family open-set; and does threat-flagging follow as a corollary.

The reframe that makes this worth doing: the earlier 'characterisation, not detection' reading understated the signal. A one-class model already recognises 'normal' at ROC-AUC **0.925**, and two workloads we had called inseparable in memory separate cleanly (Cohen's d ~ **5.6**) once the right method is used -- they scored 0.0 only because a *supervised classifier* failed on them. The memory signal carries more behavioural structure than the negative implied.

The signal is **memory only** -- APF plus a new per-page magnitude metric. The disk channel from Plan 06 is retained as a supporting **ablation**, not part of the spine (we keep the thesis on its novel contribution, the memory signal). Every decisive experiment runs **offline on data we already have**, with new captures gated behind an offline pilot. The outcome is win/win: either a working behaviour recogniser with measured family cohesion, or a rigorous, error-bounded map of exactly where memory recognises behaviour and where it does

not.

> **Safety and scope (please read first)**
> This is academic, defensive research. Every reference to 'threat', 'ransomware', 'scanner', or 'encryptor' in this document denotes a SAFE SYNTHETIC SIMULATION -- a controlled workload generator that imitates a behavioural *shape* using a reversible XOR on disposable sandbox files. There is no real malware, no real encryption, no network activity, and no persistence. See RESEARCH_SAFETY_NOTICE.md and VM_executables_phase2/docs/SAFETY_MODEL.md.

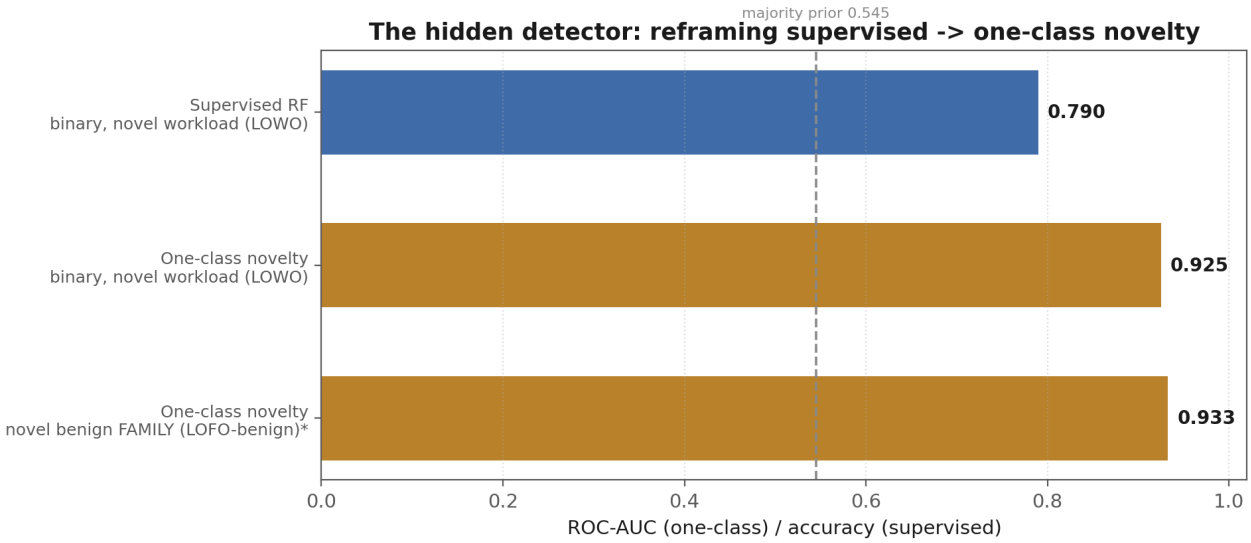
> **Dataset at a glance (current figures; grow with capture)**
> 11 workloads, 5 families, 66 cells = 11 workloads x 3 durations (120 / 300 / 600 s) x 2 repetitions. Memory snapshot cadence 500 ms; analysis window W=8, hop H=4; 5--244 windows per cell. Binary majority prior 0.545; family majority prior 0.364.

2. Motivation: what led here

APF (Active Page Fraction) is the fraction of 4 KB guest memory pages that change between two consecutive snapshots -- one number per snapshot, measured host-side with no guest agent. Plans 01--04 built and locked the capture pipeline; Plan 05 relieved a degrees-of-freedom starvation problem and ran a full 66-cell campaign; Plan 06 added a second, disk-I/O channel.

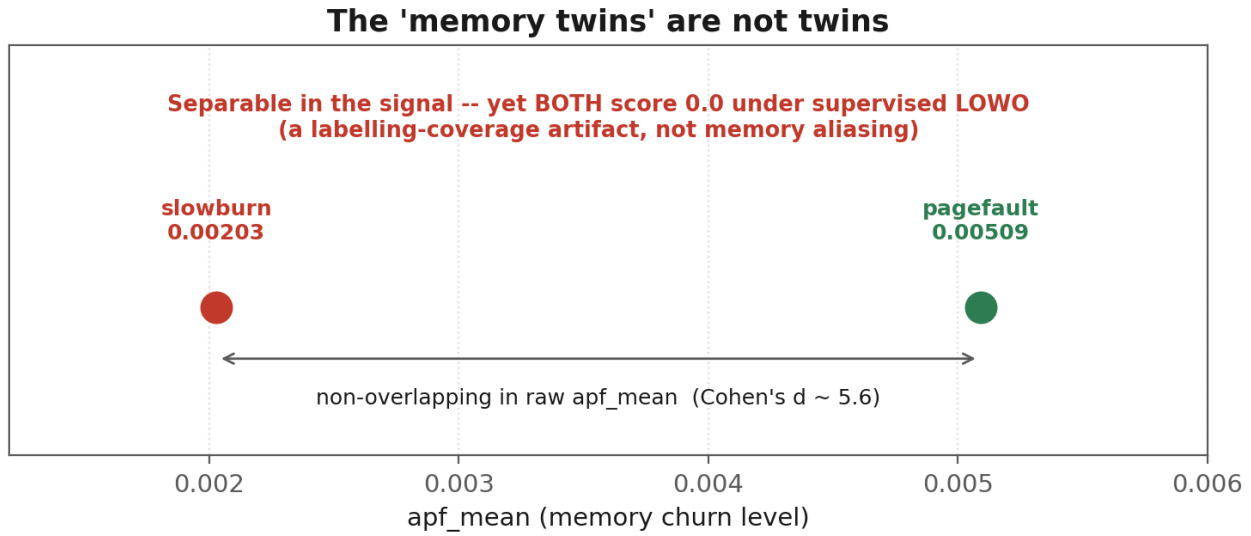
The downstream conclusion was sobering and honest: a **supervised** threat/benign classifier did not generalize to unseen workloads or families (binary leave-one-workload-out ~0.79, leave-one-family-out ~0.58, and 0.0 on the stealth pair), and a second, disk-I/O channel helped **characterization** (instance identification rose to 0.985) but not detection. Hence the earlier reading: characterization, not detection. (Plan 07 keeps the spine **memory-only** and treats that disk channel as a supporting ablation -- see Metrics.)

The review then noticed what the supervised framing had hidden. The **same committed results file** contains a one-class novelty detector at ROC-AUC 0.925 / 0.933, and the 'memory twins' separate cleanly in raw apf_mean. In other words, the negative result was a property of *how we asked the question* (a supervised boundary across too few workloads), not of the memory signal. Plan 07 builds on that: a memory-only behaviour recogniser, with detection as a corollary.



** LOF LOFO-benign 0.933 flagged INVALID (k=10 neighbours > 6-cell app family); re-run with k capped.*

The motivating result. A one-class novelty detector already beats the supervised classifier the thesis used to declare detection dead. The 0.933 is flagged for a fixable validity issue (see Methodology).



The 'memory twins' are not twins: slowburn and pagefault are non-overlapping in raw apf_mean, yet the supervised classifier scores both at 0.0 under leave-one-workload-out -- a labelling-coverage artifact.

3. Research questions

The questions group into three themes, **led by the recogniser** -- the centrepiece. Detection appears last, as a corollary. None of the eight is assumed; every outcome is a publishable finding.

Theme A -- The recogniser: characterisation, families, novelty (the centrepiece)

- **RQ1 -- Are families cohesive?** Do the workloads of a family resemble each other in memory enough for a single 'family-normal' model to represent them? Cohesion is **measured** per family, not assumed.
- **RQ2 -- Does a family-normal model generalise?** Trained on some members of a family, does it accept a held-out, *unseen* member and reject other families?
- **RQ3 -- Does memory respect our taxonomy?** With no labels, do data-driven clusters of the memory signal recover the families we named, or carve the space differently?
- **RQ4 -- Can we flag the novel, open-set?** Held out a whole family, does the system label it **NOVEL** rather than forcing it into a known family -- and does a per-family one-class *bank* beat a discriminative classifier with a *reject option* (or tie at this sample size)?

Theme B -- The signal

- **RQ5 -- Do magnitude-aware metrics catch stealth?** Does mean-Hamming-per-changed-page expose the simulated encryptor (slowburn) that APF's changed-page count cannot see?
- **RQ6 -- How cheap can capture be?** What is the minimum useful trace length, and how coarsely can we sample, before the signal degrades -- which sets the budget for expanding the workload set?

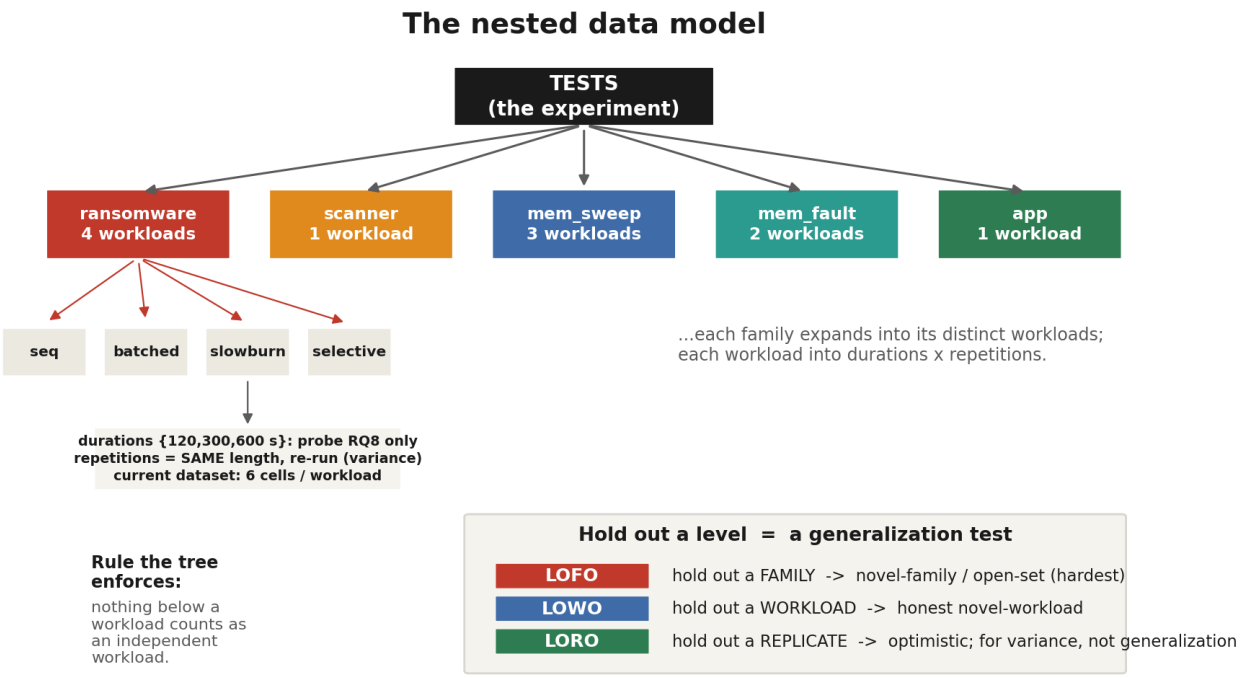
Theme C -- Detection, as a corollary of the recogniser

- **RQ7 -- Does threat-flagging ride on top?** Once behaviour is recognised by family, can 'flag if it matches a known threat-shaped family' be shown to work, with error bars?
- **RQ8 -- Is the masquerade resolved in memory alone?** Do slowburn and pagefault separate in the memory signal *without* the disk channel -- confirming the spine needs only memory?

Theme A is the core. Themes B and C support and extend it; detection (Theme C) is deliberately downstream of recognition, not the headline.

4. How we look at the data

The experiment is built on a **nested data model**. Each level is both a statistical unit and a cross-validation protocol, and the structure enforces the rules that protect us from false confidence.



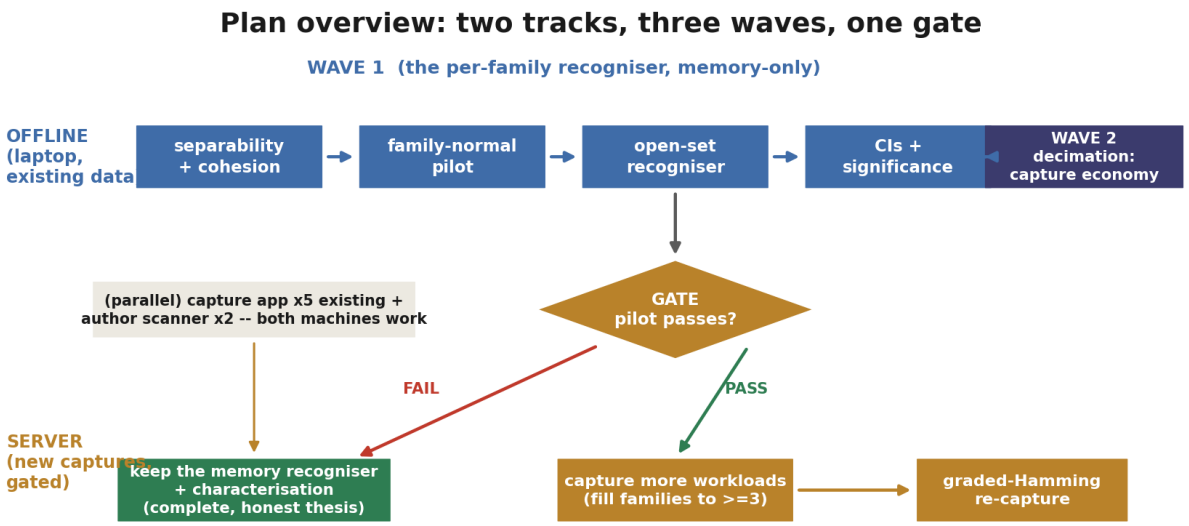
The nested data model and how holding out each level maps to a cross-validation protocol of increasing honesty.

Duration and repetition are different axes, and the distinction matters. A **repetition** is the *exact same workload at the same length, run again* -- it differs only through the signal's stochasticity (scheduler, memory layout, cache, host noise), so repetitions characterise within-workload variance. **Duration** (120 / 300 / 600 s) is a separate, controlled factor that exists in the current dataset only to answer RQ8 -- how short a capture can be. The go-forward design is therefore: use the duration sweep *once* to fix a working length, then capture **many repetitions at that single fixed length**. The current 3-durations x 2-repetitions structure is the RQ8 probe, not the permanent shape of the data.

The binding rule the tree enforces: **nothing below a workload counts as an independent workload.** A generalization claim must hold out a whole workload (LOWO) or a whole family (LOFO); repetitions and analysis windows feed variance estimation only. (All cell and family counts in this document are current figures and grow as we capture.)

5. Plan overview

Work proceeds on two tracks in parallel -- offline analysis on data in hand, and (gated) new captures on the server -- across three waves. A single decision gate governs whether any server time is spent. The whole plan is **memory-only**.



schematic

Two tracks, three waves, one gate. Wave 1 (offline) builds the per-family recogniser from the memory signal; the gate releases server captures only if the offline pilot passes; either gate outcome leaves a complete, honest thesis.

Wave 1 (offline, existing data) -- build the recogniser

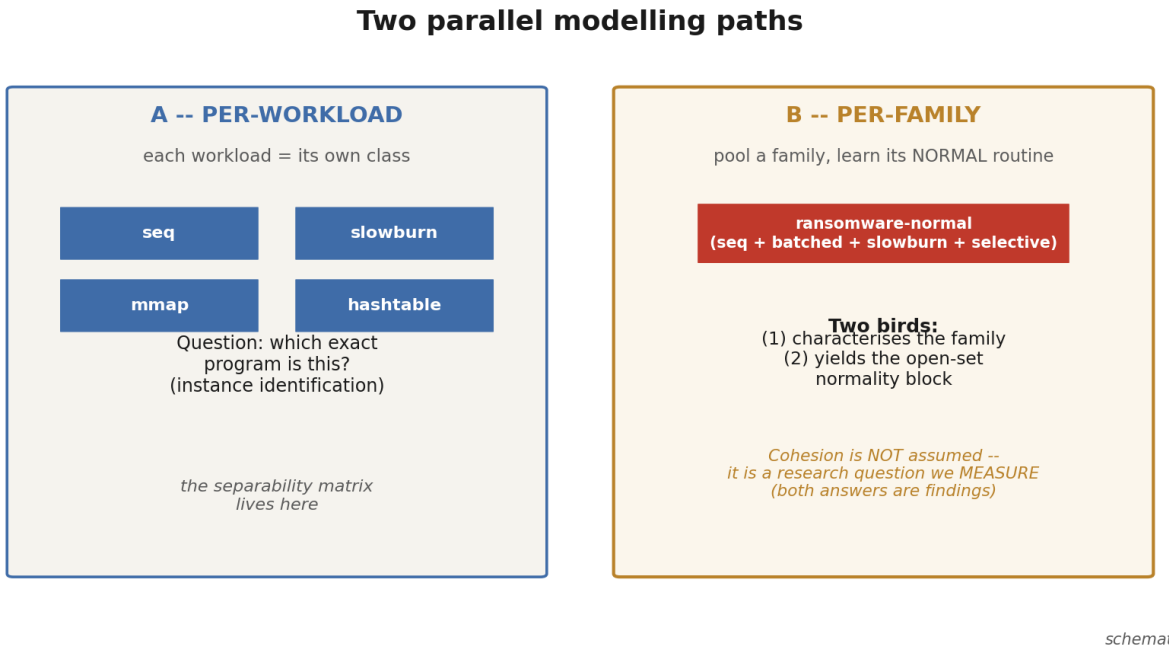
- **Separability + cohesion** -- the workload-level matrix, blocked by family: which behaviours alias, and how cohesive each family is (RQ1, RQ3).
- **Family-normal pilot** -- on the families that already have enough workloads (ransomware, mem_sweep), train a family-normal model and test it on a held-out, unseen member (RQ2); run the bank vs reject-option bake-off (RQ4).
- **Open-set recogniser** -- hold out a whole family, check it is flagged NOVEL; report a leak-free operating point and per-workload recall.
- **Confidence intervals + significance** on every headline number (workload / family unit).

Wave 2 (offline) and Wave 3 (server, gated)

- Wave 2: the **capture-economy** study -- trajectory decimation and truncation -- fixing the minimum useful cadence and length (RQ6). The Plan 06 disk experiments are dropped from the spine.
- Wave 3: **capture more workloads** to lift the small families and add the absent IO and IDLE families, and (if justified) a graded-Hamming re-capture -- only after the gate passes.

6. Two modelling paths

Two distinct, complementary modelling paths run in parallel. The **per-family path is the centrepiece** (the recogniser); the per-workload path supports it with instance-level separability. They are not competitors.

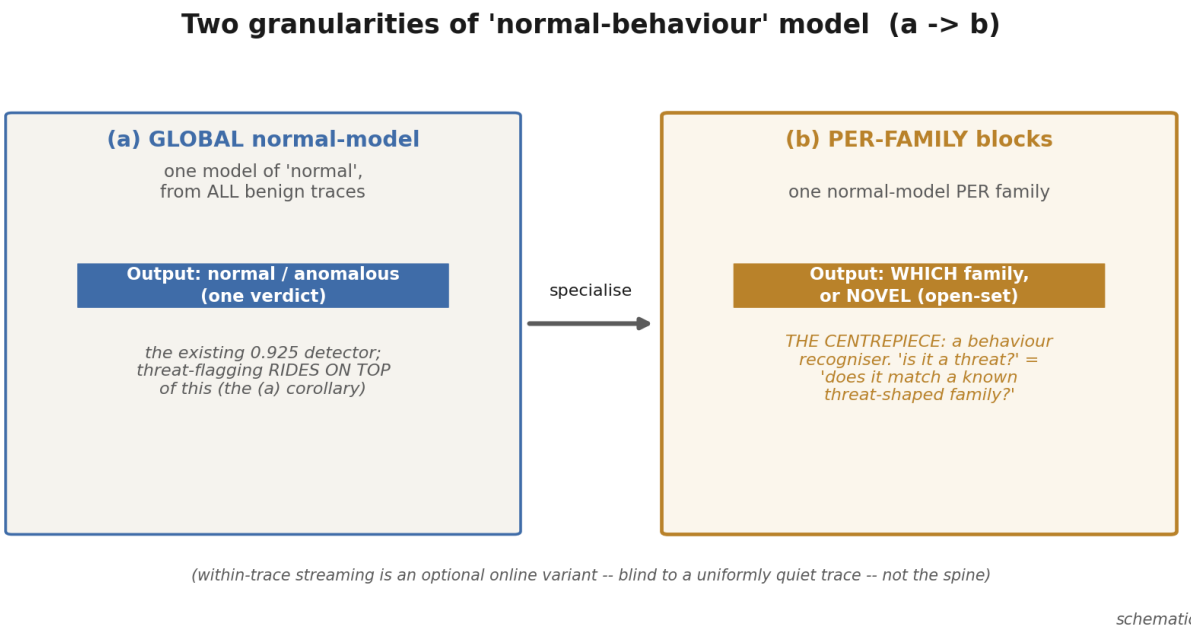


The per-workload path (instance identification) and the per-family path (learn a family's normal routine). The per-family path both characterises the family and yields an open-set normality model.

Cohesion is a research question, not an assumption. Pooling a family 'as one' only describes it well if its workloads resemble each other -- and whether they do is itself a finding (RQ3). If a family is cohesive, its label maps to a real shared memory signature. If it is not (an early data point: ransomware's leave-one-family-out recall was only 0.25), then the semantic family does **not** correspond to a single memory routine, which is a stronger and more interesting result -- it forces sub-family granularity. We quantify cohesion as a number (a between-family / within-family ratio plus within-family generalization) and also test it without labels: does clustering the memory signal recover the families we imposed?

7. Global model, then per-family blocks

All the models here are **novelty / one-class** -- learn what 'normal' looks like, flag what does not fit. They differ in **granularity**, and that granularity is the (a) -> (b) structure of the whole plan.



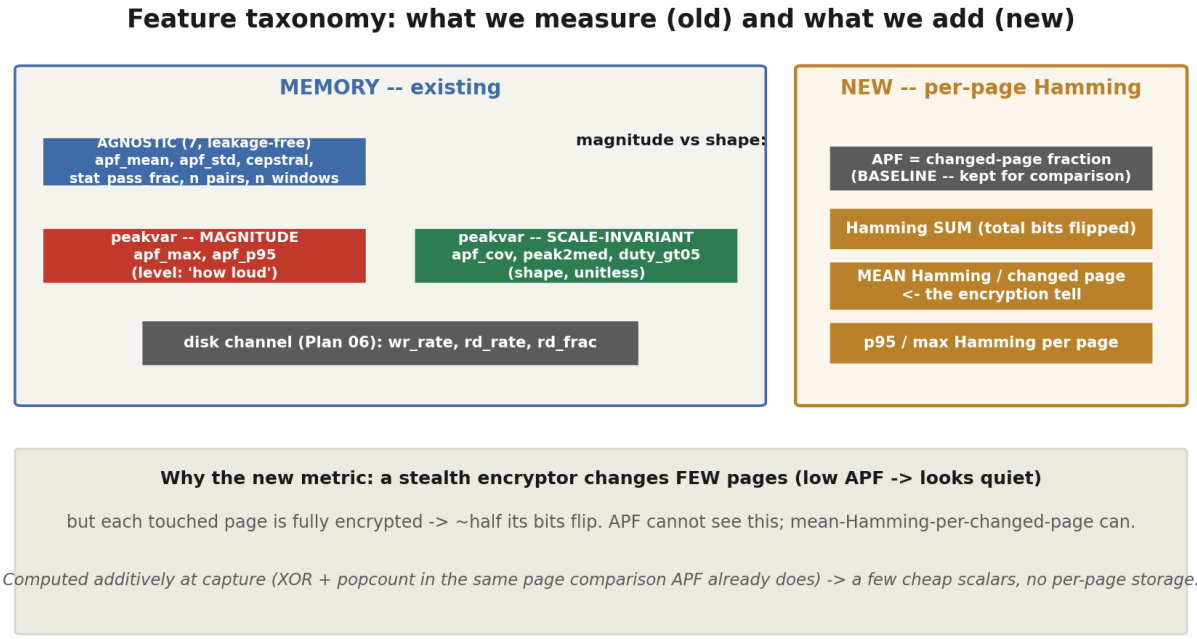
(a) one GLOBAL normal-model (output: normal / anomalous -- the detection corollary) specialises into (b) one normal-model PER FAMILY (output: which family, or NOVEL -- the recogniser, and the centrepiece). Within-trace streaming is an optional online variant, not the spine.

For the per-family blocks (b), one mechanism question is settled **empirically**, not by argument: a **per-family one-class bank** (one density model per family) versus a **discriminative classifier with a reject option** (open-set with two thresholds rather than five blocks). At the current sample size the two may be statistically indistinguishable, so the pilot runs **both on the same folds** and lets the data decide (RQ4).

> **The sharp failure mode to guard against**
> The per-family rule 'clear it if it matches a known benign family' can launder the stealth simulation: slowburn (low APF) is closest to a benign family in a magnitude-driven space, so a benign block would clear it. Guard: run the pilot on magnitude-invariant shape features (apf_cov, peak2med, duty_gt05) and hold slowburn out as a probe, with its recall on the headline table so it can never hide in an average.

8. Metrics: existing and new

The features split into level (magnitude) and shape (scale-invariant). The strong one-class model leans on magnitude features, which is also its weakness: magnitude is a poor proxy for the threat-shaped class here (the stealth simulation slowburn is *quieter* than the benign mmap is loud). Plan 07 therefore tests magnitude, scale-invariant, and combined feature sets, and adds a new magnitude-aware-but-cheap metric. The recogniser uses **memory features only**; the disk features remain in the inventory as a second-channel **ablation**, not part of the spine.



The feature taxonomy. Existing memory and disk features (left) plus the new per-page Hamming scalars (right), with the motivation for the new metric.

Metric	Group	Status	What it captures
apf_mean, apf_std, cepstral, stat_pass_frac, n_pairs, n_windows	AGNOSTIC (7)	existing	leakage-free level + periodicity + quality
apf_max, apf_p95	peakvar -- magnitude	existing	level / loudness of churn
apf_cov, peak2med, duty_gt05	peakvar -- scale-invariant	existing	shape / burstiness (unitless)
wr_rate, rd_rate, rd_frac	disk (Plan 06)	ablation only	a second-channel comparison -- NOT part of the memory-only spine
apf, n_changed	depth -- APF baseline	kept	**breadth**: how many pages changed (the existing signal, kept beside the new axes for comparison)
ham_mean, ham_max, ham_p95, ham_std	magnitude -- per-page	NEW (additive)	**depth distribution**: bits flipped per changed page -- the *magnitude distribution* (max = the encryption tell)
ham_sum, ham_frac	magnitude -- per-snapshot	NEW (additive)	**depth, global**: total bits flipped this snapshot (and as a fraction of all memory bits)
ent_mean, ent_max, ent_p95, ent_std	entropy -- per-page	NEW (additive)	**randomness distribution**: per-page byte entropy -- the *entropy distribution* (max = a page that looks encrypted)
ent_pooled	entropy -- per-snapshot	NEW (additive)	**randomness, global**: entropy of all changed bytes pooled into one histogram

*Feature inventory. APF (breadth) is kept as the baseline; the new **depth axes** -- weighted Hamming (magnitude) and byte entropy (randomness), each as a per-page distribution and a per-snapshot global value -- are computed additively inside the page comparison APF already performs (one XOR + popcount + a byte histogram, on changed pages only). A few scalars per snapshot, no per-page storage. Full design and single-pass algorithm: magnitude_entropy_spec.md. The depth section below gives the 2x2 view, the equations, and a Q&A.*

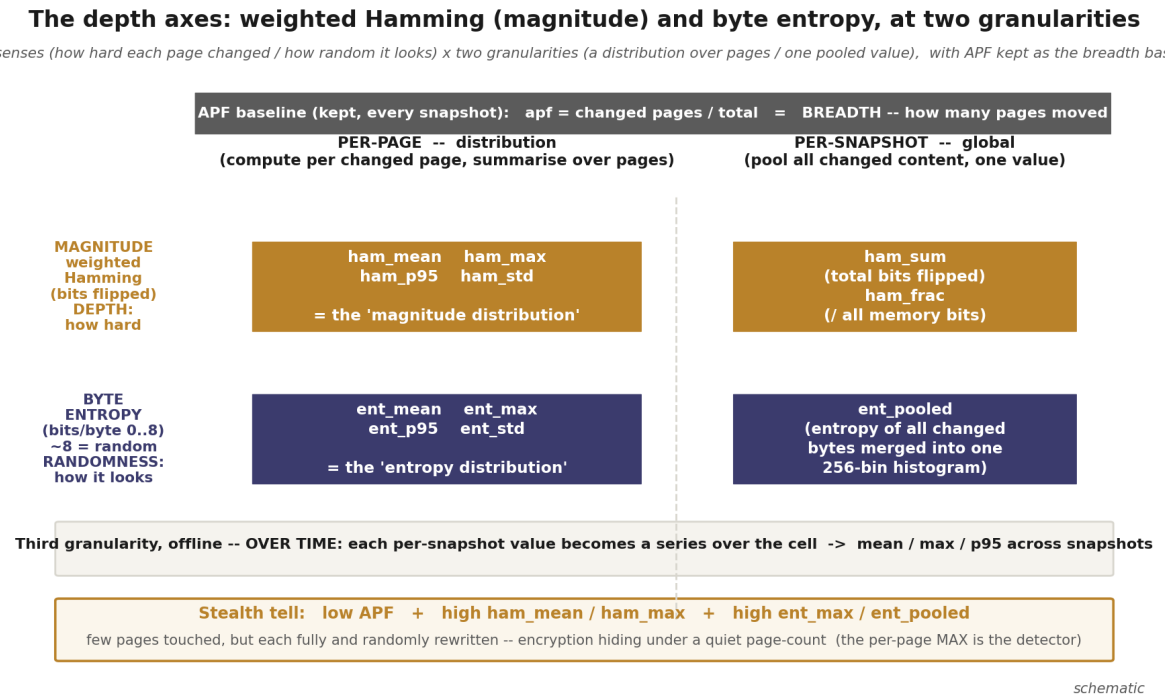
> **Why the new metric, concretely**
> APF asks only 'did this page change (yes/no)'. A stealth encryptor changes few pages (low APF, looks quiet) but fully encrypts each one (~half its bits flip). mean-Hamming-per-changed-page stays high even when the page count is low -- it can see what APF cannot. It is stored next to APF (the baseline) so the comparison is built in, and it costs a few scalars per snapshot, not per-page storage.

9. The depth axes: magnitude, entropy, and two granularities

APF measures only **breadth** -- how many pages changed between two snapshots. It says nothing about *how hard* each page changed or *what the new content looks like*, and a stealth encryptor exploits exactly that blind spot: it rewrites few pages (low APF, looks quiet) but fully encrypts each one. Plan 07 therefore adds two **depth axes** alongside APF, each measured at two granularities. APF is not replaced -- it is kept, in the same per-snapshot record, as the breadth baseline the depth axes are compared against.

Magnitude (weighted Hamming). For each changed page we XOR it against its previous copy and count the set bits: h = popcount(prev XOR curr) = the number of bits that flipped. This *weights* each changed page by how hard it changed, where APF gave every changed page an equal yes/no vote. Summed over the snapshot it is the total bit-churn; spread over the changed pages it tells us whether change is thin and uniform or concentrated in a few heavily rewritten pages.

Byte entropy. For a changed page we histogram its 256 possible byte values and compute Shannon entropy, H = -sum p_b log2 p_b, in the range 0--8 bits per byte. Near 8 the bytes are uniformly distributed -- the fingerprint of compressed or encrypted data; nearer 4--5 the content is structured. This is the encryption tell that magnitude alone cannot give: a page can flip many bits and still be structured, but a page that flips many bits *and* looks random is the encryptor's signature.



The depth block at a glance. Two senses (magnitude, byte entropy) at two granularities -- a distribution over changed pages, and a single pooled value per snapshot -- with APF kept as the breadth baseline. The per-page max is the stealth detector; the pooled values and APF are the bulk views. (Schematic of the feature layout, not data.)

Axis (what it senses)	Per-page -- distribution over the changed pages	Per-snapshot -- one pooled value
APF -- *breadth*: how many pages moved. The baseline, kept beside the depth axes.	-- (APF is already a per-snapshot fraction; no per-page form)	**apf** = changed pages / total; **n_changed**
Magnitude -- *weighted Hamming*: how hard each page changed. Per page, h = popcount(prev XOR curr) = bits flipped.	**ham_mean, ham_max, ham_p95, ham_std** -- the *magnitude distribution* across changed pages	**ham_sum** (total bits flipped), **ham_frac** (bits flipped / all memory bits)
Entropy -- *byte randomness*: how random the changed content looks (bits/byte, 0--8; ~8 = ciphertext-like).	**ent_mean, ent_max, ent_p95, ent_std** -- the *entropy distribution* across changed pages	**ent_pooled** -- entropy of all changed bytes merged into one 256-bin histogram

*The depth block: two senses (magnitude, entropy) at two granularities, with APF the breadth baseline alongside. **Per-page** = compute on each changed page, then summarise the spread over pages (a distribution: mean / max / p95 / std). **Per-snapshot** = pool everything into one value. A single pooled value has no within-snapshot mean / max -- those reappear **offline**, taken over the cell's snapshots (a third, over-time granularity that applies to every column). The per-page **max** is the stealth detector: one fully-encrypted page among many quiet ones spikes ham_max / ent_max while the pooled value and APF stay low.*

Why two granularities -- and where 'over time' comes in

The two columns are two ways to collapse a snapshot's changed pages into numbers. The **per-page** column computes the metric on each changed page and then summarises the spread *over pages* -- mean, max, p95, std -- which is why we call it a *distribution*. The **per-snapshot** column instead pools all changed content into one blob and computes a single number. They are complementary: the per-page max catches a single encrypted page that the pooled value would average away, while the pooled value captures the overall composition of everything that changed. There is also a third, implicit granularity that applies to every column -- **over time**. A cell is a whole trajectory of snapshots, so offline each per-snapshot scalar (including the single pooled values) becomes a time series, summarised again with mean / max / p95 across the cell. That is

where the mean and max of the pooled values come from; within one snapshot a pooled value is necessarily single.

> **The common confusion: ent_pooled vs ent_mean / ent_max**
> ent_pooled is *not* the average of the per-page entropies. It pools every changed byte into one histogram and takes entropy once -- a single global number, the entropy twin of ham_sum. The per-page entropy mean and max are separate scalars, ent_mean and ent_max, computed on each page first and then summarised over pages. Use ent_mean for 'the typical page's randomness', ent_max for 'the most random page', ent_pooled for 'the randomness of the changed memory as a whole'. All three are kept; the same distinction holds for ham_mean / ham_max versus ham_sum.

The equations

Magnitude, per changed page: **h = popcount(prev XOR curr)** (bits). Byte entropy, per changed page, in the count form we actually compute (it avoids per-bin division and reuses a precomputed table T[k] = k log2 k): **H = log2(B) - (1/B) sum_b c_b log2(c_b)**, where B = 4096 bytes per page and c_b is the count of byte value b on that page. The pooled entropy uses the same formula on the summed histogram C_b = sum-over-pages of c_b, with total B*K over the K changed pages. One page-diff feeds APF, the popcount, and the histogram together -- the full single-pass algorithm is in magnitude_entropy_spec.md.

Questions and answers

The questions that came up while designing the depth axes, and the ones most likely next:

Question	Answer
Do we still compute APF, or only the new metrics?	Both, every snapshot. APF is kept as the breadth baseline and sits in the same record next to the depth axes, so the comparison 'what do magnitude / entropy add over APF?' is built in. Not one or the other.
What is the 'weighted Hamming' (magnitude)?	For each changed page, h = popcount(prev XOR curr) = how many bits flipped. 'Weighted' because each page contributes *how hard* it changed, not a yes/no vote. APF counts changed pages; magnitude weighs them by bit-distance.
What is byte entropy?	For a changed page, histogram its 256 possible byte values and compute H = -sum p_b log2 p_b, in 0--8 bits/byte. Near 8 the bytes look uniform -- the fingerprint of compressed or encrypted data; nearer 4--5 the content is structured.
Per page or per snapshot?	Both -- two granularities. Per-page: compute on each changed page, then summarise across pages (mean / max / p95 / std) = the distribution. Per-snapshot: pool everything into one value. And offline, every per-snapshot value is summarised over the cell's snapshots (a third, over-time view).
What is ent_pooled, and why no mean / max on it?	It pools every changed byte this snapshot into ONE histogram and computes entropy once = one number ('overall randomness of everything that changed'). A single blob has nothing to average within a snapshot. The entropy mean / max you'd want are ent_mean / ent_max (the per-page row). ent_pooled is the global twin of ham_sum.

Question	Answer
Where is the 'magnitude distribution'?	It is ham_mean, ham_max, ham_p95, ham_std -- bits-flipped computed per changed page, then summarised over pages. That set IS the magnitude distribution. ham_sum / ham_frac are the global total beside it.
Why per-page at all -- why not just pool everything?	Pooling dilutes localised encryption. One fully-random page hidden among many quiet ones barely moves the pooled value or APF, but spikes the per-page max (ham_max, ent_max). The per-page max is the stealth detector; the pooled value is the bulk / composition view. Different failure modes, both kept.
Does count(b) count bytes in a page or across the snapshot?	Per page (4096 bytes) for the per-page layer. The pooled layer's histogram is simply the sum of the per-page histograms (K x 4096 bytes), so both granularities come from the same one pass.
Do we normalise so the probabilities sum to 1?	Conceptually yes -- p_b = count_b / total. In code we use the equivalent count form H = log2(B) - (1/B) sum_b c_b log2(c_b), which avoids per-bin division and reuses a precomputed table T[k] = k log2 k; identical result.
Why capture all of this -- isn't it a lot of features?	Capturing is the expensive, irreversible part (it needs the server); computing extra scalars from pages we already compare is nearly free (shared XOR, summed histograms). So we capture richly and let the offline feature selection decide what carries signal. More features is a modelling-stage concern (handled by leakage controls + selection), not a reason to discard data at capture time.
Is this implemented in the capture yet?	Not yet. It is **Task C**: an additive, flag-gated emit (TIMING_MAGENT) that mirrors the Plan 06 disk block and is byte-identical when off. Until it runs, captures carry APF only. Design: magnitude_entropy_spec.md.
What does a stealth workload look like in these features?	low apf + high ham_mean / ham_max + high ent_max / ent_pooled = few pages touched, but each fully and randomly rewritten. That is the signature the sandbox_stealth_* workloads were built to produce.

The questions that came up while designing the depth axes, and the ones most likely to come up next. The metric is intentionally redundant (two senses, two granularities, plus the APF baseline) so that the offline analysis -- not a capture-time guess -- decides which scalars carry the signal.

10. Tests: what we have and what to capture

The current dataset is 11 workloads across 5 families. The families are small and uneven in their number of distinct workloads, and that -- not the cell count -- is the binding constraint on the per-family work. (These figures are provisional and grow with capture.)

Family	Kind	Distinct workloads	Cells	Per-family block validatable now?
ransomware	threat	4	24	Yes -- 4 within-family folds
mem_sweep	benign	3	18	Marginal -- 3 folds (pilot-grade)
mem_fault	benign	2	12	No -- 1 degenerate fold

Family	Kind	Distinct workloads	Cells	Per-family block validatable now?
app	benign	1	6	No -- family-of-one (memorises one workload)
scanner	threat	1	6	No -- family-of-one; and a threat family

Family composition of the CURRENT 66-cell dataset. These figures are provisional -- they grow as we capture more workloads (see the catalogue below). Size, in distinct workloads, decides whether a per-family model can be honestly validated.

But the 66-cell subset is a small slice of a much larger **designed corpus**. The next-generation specifications define ~10 behaviour families and ~39 workloads -- IDLE, MEM, IO, CPU, CACHE, THREAD, NETWORK, MIXED, APP, and the security-like family -- of which the captured 11 are only a fraction. Building and sampling this corpus on the server, in parallel with the offline analysis, is the **primary mitigation for small n**, not a side task: it lifts the family and workload counts until the leave-one-out significance floors fall below 0.05.

Behaviour family	Captured (66-cell)	Built now	Status
IDLE	0	run_idle + idle_long_baseline, idle_post_workload_recovery (NEW)	ready to capture
MEM	5	5 + random_write_pages, stride_sweep_large (NEW C)	ready to capture
IO	0	rand_rw / seq_fsync / many_files (py) + read_cache_hit, direct_write_like (NEW C)	ready to capture
CPU	0	hash_loop, matrix_mult, branch_random (NEW C)	ready to capture
CACHE	0	hot_loop, cold_scan, stride_sweep (NEW C)	ready to capture
THREAD	0	lock_contention, producer_consumer, parallel_alloc (NEW C)	ready to capture
NETWORK	0	-- (deliberately NOT built)	excluded for safety -- would need sockets; preserves the repo's zero-network-code guarantee
MIXED	0	mixed_mem_io, mixed_cpu_mem, mixed_cpu_io (NEW C)	ready to capture
APP	1	6 (sqlite x2, gzip x2, json, hashtable)	captured: 1
SECURITY-like	5	8 (4 ransom + scanner + 3 stealth)	captured: 5

*The **designed corpus**: ~10 behaviour families. The six new benign system-behaviour families (CPU, CACHE, MEM+2, IO, THREAD, MIXED) and the two new IDLE scripts are now **built and smoke-tested** (16 new C binaries + 2 shell scripts), so the corpus they grow toward is ready to capture on the server -- this is what dissolves the small-n constraint. The NETWORK family is **deliberately excluded**: it would require sockets, and keeping zero network code in the workload tree is a standing safety guarantee (see SAFETY_MODEL.md). All new workloads are benign benchmarks: no network, no persistence, file I/O confined*

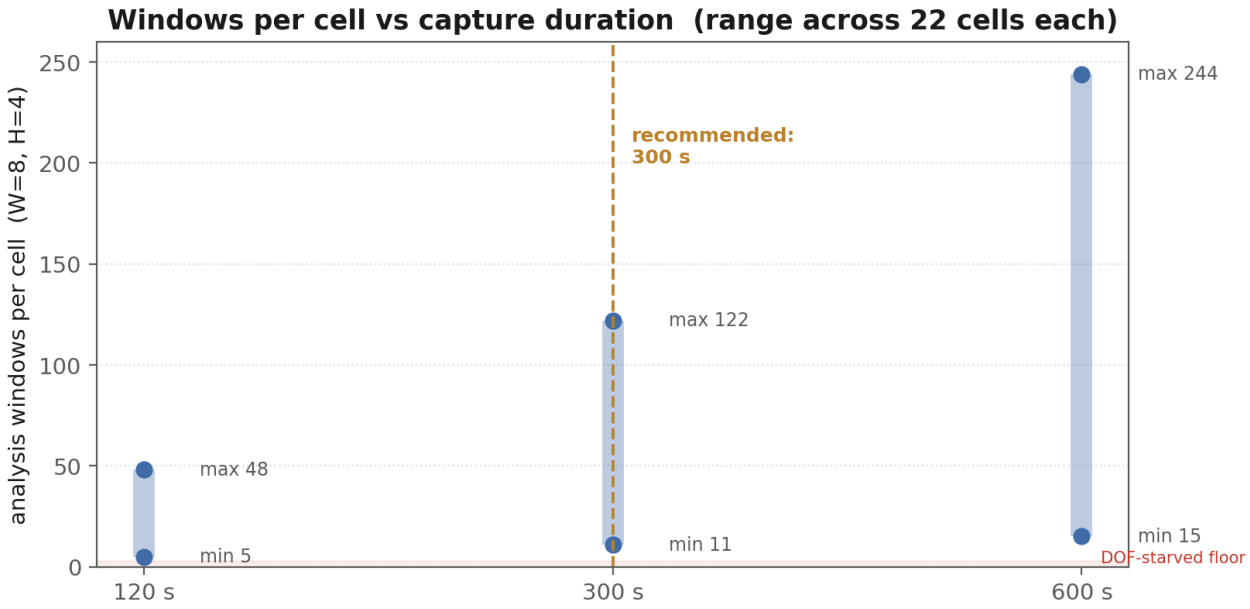
to a sandbox-validated scratch dir.

Because server access is available, we capture broadly. The priority axis is **distinct workloads** (which test generalisation); **repetitions at a fixed length** are the second axis (which estimate variance). Most of the expansion is **capture-only** -- the executables already exist -- so it costs server time, not new code; only the scanner family needs new programs written.

Priority	Action	Effect
1 -- no code	capture the built-but-uncaptured workloads: all 6 APP, the 3 phase-1 IO, the phase-1 MEM set, run_idle	adds the IO family and several APP / MEM / IDLE members immediately
2 -- author	build the specced families (CPU, CACHE, THREAD, NETWORK, MIXED) and the missing IDLE / IO / MEM variants	takes the corpus to ~10 families / ~39 workloads
3 -- author	additional security-like (scanner) variants	lifts the only size-1 threat family
ongoing	repetitions at the fixed length for every workload	within-workload variance for the bootstrap

Capture runs continuously on the server while the offline analysis proceeds on existing data. As families pass ~7 and workloads pass ~20, the leave-one-family-out / leave-one-workload-out significance floors fall below 0.05 and the descriptive findings become testable.

On duration: even 120 s already clears the analysis-window floor, so short captures are viable. We recommend **300 s** as the working length (a good window yield at half the cost of 600 s), confirm the true minimum offline by **truncating** existing 600 s traces, and thereafter capture repetitions at that single fixed length.



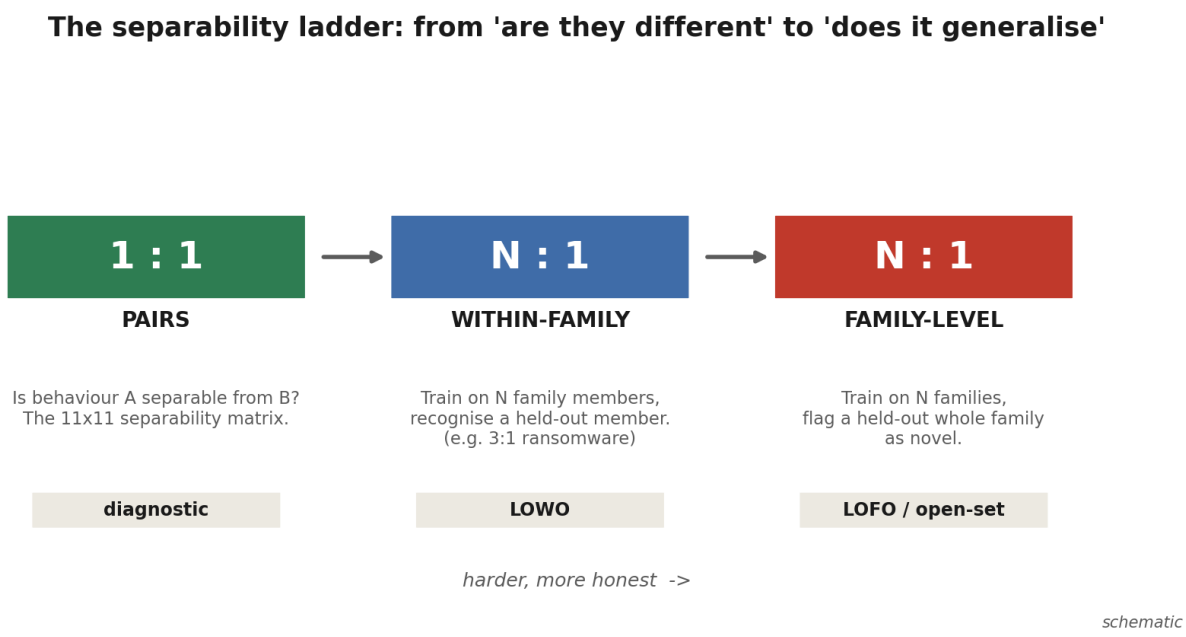
Windows per cell versus capture duration (measured). Even the shortest duration clears the starvation floor; 300 s is the recommended balance.

Duration	Windows per cell (range, W=8/H=4)	Note
120 s	5 -- 48	already clears the DOF floor
300 s	11 -- 122	recommended -- balances window count vs server time
600 s	15 -- 244	most windows, but twice the capture time

Window yield by duration, measured across the existing cells.

11. The separability ladder

Separability is examined as a ladder of increasing difficulty. The master artifact is a **workload-level separability matrix ordered by family** -- the family-level view is derivable from it by averaging blocks, but not vice versa.



From 1:1 pairs (are two behaviours separable) up through N:1 crosses (does a group generalize to a held-out member / family). The N:1 rungs are exactly the leave-one-out generalization tests.

The master artifact (schematic). Diagonal blocks measure within-family cohesion; off-diagonal blocks measure between-family separation; a cross-family low cell (green) is a masquerade. Run three times -- magnitude, scale-invariant, and combined -- to map which feature type resolves which pair.

A caution carried from the metrics section: per-trace normalization removes the absolute level, but it was the *level* that separated slowburn from pagefault. So magnitude and shape are likely **complementary** -- each resolves different pairs -- and the matrix is the instrument that shows, pair by pair, where each matters.

12. Methodology and statistical protocol

The design's credibility rests on a few non-negotiable rules drawn from the review.

- **Leak-free fitting.** The imputer, scaler, and any operating-point threshold are fit **inside each fold** on training-benign only. No global fit survives for any threshold/PR metric (the committed code fits these globally -- valid for ROC ranking, but a leak for an operating point).
- **Honest units.** Bootstrap confidence intervals resample whole **workloads** (11) for LOWO/recogniser claims and **families** (5) for LOFO; the (workload, duration) cluster bootstrap is a labelled sensitivity bound only, never the headline CI.
- **Significance by permutation/exact tests**, computed at the generalization unit -- not asymptotic tests, given n <= 11.
- **A regression anchor.** Reproduce the committed 0.925 / 0.933 before changing anything, then show what each fix moves.

A structural limit must be stated honestly: with few independent units, some comparisons **cannot** reach significance regardless of the effect. These are reported descriptively, with confidence intervals, and never dressed up with a p-value.

Independent units n	Sign / McNemar floor p = 2^(1-n)	Can reach p < 0.05?
5 (LOFO, current families)	0.0625	No, ever -- on the current 5 families
7+ (LOFO, after capturing IO + IDLE)	0.016 or lower	Yes -- the capture lifts it below 0.05
11 (LOWO, current workloads)	0.00098	Yes (needs >= 6 concordant)

*Structural significance floor (sign / McNemar). With only **5** families, leave-one-family-out cannot reach significance regardless of effect size; capturing the currently-absent IO and IDLE families (>= 7 total) drops the floor below 0.05. Few-unit comparisons are reported descriptively, with CIs, until the families grow.*

Accordingly, claims are sorted into three tiers: a small set of **confirmatory** hypotheses (the family-normal recogniser vs a permutation null, the slowburn/pagefault separability in memory, the one-class normal-model vs chance) corrected together with Holm; **descriptive** results (all leave-one-family-out comparisons) reported as intervals with no significance verdict; and **effect-size-only** results (the masquerade separation magnitude). The false-positive benefit of the per-family scheme is, on current data, demonstrable only as a worked mechanism -- with at most ~5 honest novel-benign test points, its exact-McNemar floor is 0.0625, so it cannot reach significance until the families grow.

13. Expected results

Each experiment is designed so that both outcomes advance the thesis. The table states, per experiment, what would confirm versus refute -- and what each means.

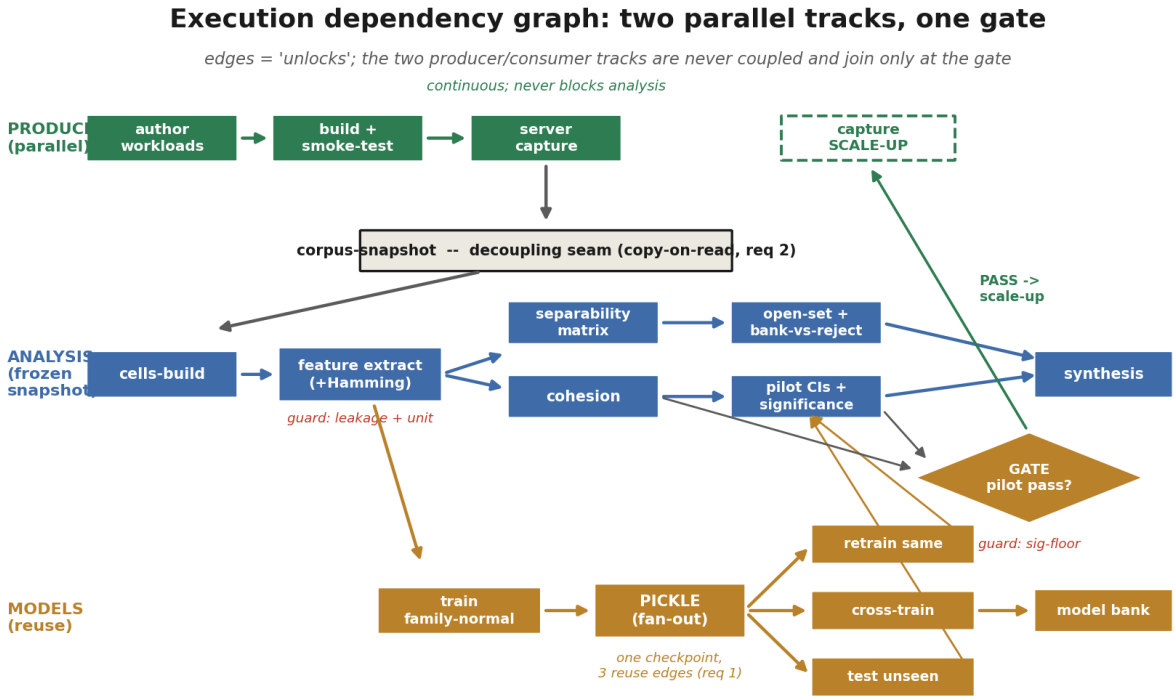
Experiment	CONFIRM (and its meaning)	REFUTE (and its meaning)
Separability + cohesion (per family, memory only)	within-family tight, between-family distinct -> families have a coherent memory signature; family-normal models are meaningful	families not cohesive -> the semantic taxonomy does not map to memory; sub-family granularity needed (still a finding)
Family-normal recogniser (within-family hold-one-out)	held-out, unseen family member accepted; other families rejected -> the recogniser generalises within a family	held-out member rejected, or cross-family accepted -> the family label does not map to one routine
Open-set novelty (hold out a whole family)	a never-seen family is flagged NOVEL, not forced into a known one -> open-set recognition works	the novel family is absorbed into a known one -> the novelty boundary is too weak
Memory resolves the masquerade (slowburn vs pagefault, NO disk)	separable in memory alone (AUC ~ 1.0) -> the disk channel is not needed for the spine	inseparable out-of-sample -> a second channel is genuinely required after all
Detection as a corollary of (b)	threat-shaped families are recognised + flag-on-match works with CIs -> detection rides on recognition	threat families are not recognisable as families -> memory recognises kinds but not threat-ness (an honest negative)

Every experiment is win/win. The spine is the per-family recogniser (b); detection (a) is derived from it.

> **Why this is win/win**
>
> If the per-family recogniser generalises and families prove cohesive, the thesis is a working **memory-only behaviour recogniser** with open-set novelty -- and detection follows as a corollary. If families prove *not* cohesive, that is itself a finding (the memory signal carves behaviour differently from our taxonomy), reported with error bars, and it redirects the capture toward the granularity that does work. Neither path is a dead end.

14. Execution dependency graph (the checklist)

The program is organised as a **dependency graph, not a schedule**: an edge means one step *unlocks* another, and steps with no edge between them run in parallel. The graph has two **never-coupled tracks** -- a parallel producer (generate and capture the growing corpus) and a consumer (offline analysis on a frozen snapshot) -- joined at a single decision gate. It is meant as a checklist: at any moment you can see what is unblocked and what may proceed together.



The dependency graph. Producer track (top, green) runs continuously and never blocks analysis; the corpus-snapshot is the copy-on-read seam that decouples the two tracks; the analysis track (blue) consumes a frozen snapshot; the models track (gold) turns one trained family-normal into a reusable pickle with three fan-out reuse edges; the gate joins them and gates only the large capture scale-up.

Three structural properties

- **Generation is a parallel, non-blocking producer.** Authoring and capturing new workloads feeds the corpus but has no edge back into analysis; analysis pins one immutable snapshot, so generation can keep appending without disturbing any in-flight analysis. Neither side ever blocks the other.
- **Trained models are reusable artifacts, not one-shot results.** A fitted family-normal is serialised once into a first-class *pickle* node, from which three non-destructive, parallel reuse edges open -- retrain on the same family, cross-train against others, test on unseen data. Many checkpoints with kept lineage, not one model.
- **The graph is acyclic and parallelism is explicit.** Even the retrain 'loop' is acyclic: each refit is a new registry entry pointing back at its parent (a data attribute), not an edge into the parent node. Three guardrails (leakage, unit-of-analysis, significance floor) sit *on* edges and must be clean before the edge is traversed.

The checklist below is the graph read as stages. Each stage names the nodes you can do in parallel once its prerequisite is met.

Stage	Unlocked once...	Do in parallel
1	start	pin provenance (git / qemu / vm-image / sklearn versions)
2	provenance pinned	author workloads (producer) AND stand up the model registry
3	workloads authored	build + smoke-test; server capture (continuous, non-blocking)
4	cells captured (or the existing 66)	validate cells (schema / provenance); freeze the corpus-snapshot (seam)
5	snapshot pinned	build the cells-dir (sets unit of analysis = workload)
6	cells built	extract the one feature matrix (+Hamming) -- guards leakage + unit are BLOCKING here
7	features ready	separability matrix; train the family-normals; train the reject-option classifier
8	family-normal trained + registry up	pickle the models (the fan-out node)
9	models pickled	reuse edges: retrain-same-family; cross-train-other; test-on-unseen
10	pickles + cross-train done	compose the per-family pickles into the model bank
11	reuse edges + bank + reject ready	measure family cohesion; open-set novelty + bank-vs-reject bake-off
12	test-unseen + open-set done	pilot CIs + permutation significance -- guard sig-floor is BLOCKING into the gate
13	cohesion + pilot CIs + pickles done	THE GATE: pass -> authorise capture scale-up; fail -> stop at the recogniser + characterisation
14	all analysis + gate done	synthesis / writeup

*The checklist. Each stage lists the nodes you can tick off **in parallel** once its prerequisite is met. The two producer/consumer tracks run concurrently throughout; on the existing 66 cells the offline chain (stages 4-14) can start immediately while the server captures the corpus in the background. The gate (stage 13) gates only the large CAPTURE scale-up -- it never gates generation.*

> **Critical path (the longest required chain)**
> pin provenance -> server capture -> validate -> corpus-snapshot -> cells-build -> feature extract -> train family-normal -> pickle -> test-unseen -> open-set -> pilot CIs -> GATE -> synthesis. Everything off this chain (the separability matrix, cross-training, the reject classifier, generation) runs in parallel beside it.

Model reuse (req 1): one pickle, many uses

The pickle node is content-addressed by a model_id (hash of training code + feature matrix + the sorted training cell ids + the family split + hyperparameters). From it: (a) retrain on more same-family traces yields a *child* checkpoint (parent_model_id set, so lineage chains stay acyclic); (b) cross-training against other families is pure inference and produces the off-diagonal reject matrix; (c) test-on-unseen serves both within-family generalisation (held-out member) and open-set novelty (held-out whole family). The bank is a composite over the per-family pickles.

Five questions the gate must confront

(1) Can family cohesion even be measured with a CI clearing the 0.545 prior at n=3-4 workloads, or is the pilot 'inconclusive' and a small targeted capture needed before a real gate decision? (2) A low-APF / high-Hamming stealth family now exists (sandbox_stealth_microwrite / scattered / paced); capture it early so the Hamming axis can be tested at the pilot. (3) Open-set ROC over 5 families (2 testable) may be anecdotal until the corpus reaches ~10. (4) The agnostic feature whitelist needs its own leakage audit -- feature choice must be independent of family labels. (5) Capture-economy decimation must not break the pilot's significance floor: measure the economy curve on full-length data before scaling up on shorter traces.

When the gate is inconclusive: a pre-gate cohesion capture

The gate is not strictly pass/fail. With only n=3-4 workloads in the two testable families, the cohesion confidence interval may be too wide to clear the 0.545 prior in either direction; the honest verdict is then **inconclusive** -- a third outcome beside pass and fail. The remedy is a **small, bounded, targeted capture before the gate**, not after: add a few distinct workloads to the families that already have >=3 members (ransomware, mem_sweep), just enough to lift n until cohesion is measurable with a confidence interval. This pre-gate capture is tiny and specific -- distinct from the large post-gate scale-up -- and it converts an unanswerable gate into a real decision. The gate therefore sequences as: measure cohesion; if inconclusive, run the small targeted capture and re-measure; only then decide. A pre-gate capture is cheap insurance against spending the large scale-up on a verdict the pilot could never have reached.

15. Risks and honest scope

- **Small n -- a current state, not a fixed limit.** The *offline* analyses are bounded by today's 11 workloads / 5 families, so several current findings are descriptive rather than significant. This is **actively mitigated**, not accepted: the parallel capture track grows the corpus toward the designed catalogue of ~10 behaviour families and ~39 workloads (the next-generation specifications in VM_executables), at which point leave-one-family-out (~0.002 floor at 10 families) and leave-one-workload-out fall well below 0.05 and the descriptive findings become significance-capable.
- **Synthetic workloads.** External validity to real malware is untested; this is a controlled study of behaviour classes, not a field detector.
- **Capture-environment confound.** New captures must use the same host configuration as the original 66, or family identity could entangle with capture conditions.
- **Cohesion may be low.** The per-family path may show that families are not cohesive in memory -- reported as a finding, not hidden.

- **Per-page spatial detail is not retained.** The new Hamming scalars keep aggregate magnitude, not which pages changed; a future spatial analysis would need a richer capture.
- **Disk is out of scope on the spine.** The Plan 06 disk-channel result is reported only as an ablation; the recogniser claims nothing from it, keeping the contribution memory-only.

The plan is deliberately staged so that the expensive, irreversible step (server capture) is the last one, and is taken only after the cheap, offline steps have shown it is worth taking.